Generative AI - Assignment No. 1

Mohammad Hamza Imran[1 [22I−0865]]

FAST NUCES, Department of Computer Science, Islamabad, Pakistan
i220865@nu.edu.pk

Abstract. This report presents a comprehensive analysis of three fundamental deep learning architectures through practical implementation and evaluation. We investigate Convolutional Neural Networks (CNNs) for image classification on the CIFAR-10 dataset, Recurrent Neural Networks (RNNs) for next-word prediction using Shakespeare text, and PixelCNN, RowLSTM and Diagonal BiLSTM models for generative image modeling. Our CNN implementation achieves 83.42% accuracy on CIFAR-10 through systematic hyperparameter optimization, demonstrating the effectiveness of deeper architectures with increased filter counts. The RNN model for text generation achieves 38.28% top-1 accuracy and 60.09% top-5 accuracy, with perplexity of 31.47 on the validation set. For generative modeling, we implement and compare PixelCNN, Row LSTM, and Diagonal BiLSTM architectures, achieving 7.8889, 7.8046, and 7.7805 bits per dimension respectively, confirming the theoretical performance hierarchy. Our ablation studies reveal the critical impact of hyperparameter selection on model performance, with learning rate, batch size, and architectural depth significantly influencing convergence and accuracy. The results demonstrate the practical application of deep learning principles and provide insights into the trade-offs between model complexity and performance across different domains.

Keywords: Deep Learning · Convolutional Neural Networks · Recurrent Neural Networks · PixelRNN · Image Classification · Text Generation · Generative Modeling · Generative AI.

1 Introduction

Deep learning has revolutionized the field of artificial intelligence, enabling breakthroughs in computer vision, natural language processing, and generative modeling. This paper presents a comprehensive study of three fundamental deep learning architectures through practical implementation and rigorous evaluation. Our investigation spans three critical domains: image classification using Convolutional Neural Networks (CNNs), sequential text generation with Recurrent Neural Networks (RNNs), and autoregressive image generation using PixelRNN architectures.

The first part of our study focuses on CNN implementation for the CIFAR-10 image classification task. CIFAR-10, consisting of 60,000 32×32 color images

across 10 classes, presents a challenging benchmark for evaluating CNN architectures. We implement a flexible CNN framework that allows systematic exploration of hyperparameters including learning rate, batch size, number of convolutional filters, and network depth. Through comprehensive ablation studies, we identify optimal configurations and demonstrate the impact of architectural choices on model performance.

The second component examines RNN-based text generation using the Shakespeare dataset. We implement a word-level LSTM model with custom embeddings, avoiding pre-trained word vectors to demonstrate the model's ability to learn representations from scratch. The model processes sequences of 20 words and generates coherent text continuations, with evaluation metrics including perplexity and top-k accuracy providing insights into the model's language understanding capabilities.

The third part implements and compares three PixelRNN architectures as described in the seminal work by van den Oord et al. [1]. We implement PixelCNN with masked convolutions, Row LSTM with sequential row processing, and Diagonal BiLSTM with bidirectional diagonal processing. Our implementation validates the theoretical performance hierarchy while providing practical insights into the computational trade-offs between these architectures.

This comprehensive study demonstrates the practical application of deep learning principles across multiple domains, providing valuable insights into model design, hyperparameter optimization, and performance evaluation. The results contribute to understanding the effectiveness of different architectural choices and their impact on model performance in real-world applications.

2 Question 1: CNN Implementation for CIFAR-10 Classification

2.1 Objective and Dataset

The objective of this task is to build and evaluate a Convolutional Neural Network for image classification using the CIFAR-10 dataset. CIFAR-10 consists of 60,000 32×32 color images across 10 classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. The dataset is split into 50,000 training images and 10,000 test images, providing a balanced benchmark for evaluating CNN architectures.

2.2 CNN Architecture and Implementation

We implement a flexible CNN architecture that allows systematic exploration of different configurations. The base architecture consists of three convolutional blocks, each containing a convolutional layer, batch normalization, ReLU activation, and max pooling. The network includes dropout regularization and fully connected layers for classification.

The model processes 32×32×3 input images through the following structure:

- Conv1: 3→32 channels, 3×3 kernel, batch normalization, ReLU, max pooling
- Conv2: 32→64 channels, 3×3 kernel, batch normalization, ReLU, max pooling
- Conv3: 64→128 channels, 3×3 kernel, batch normalization, ReLU, max pooling
- Fully connected layers: 128×4×4→512→256→10 with dropout (0.5)

The model contains 1,276,682 trainable parameters and uses Adam optimizer with learning rate scheduling for training.

2.3 Training and Evaluation Results

The baseline CNN model achieves 80.06% accuracy on the test set after 20 epochs of training. Training curves show consistent convergence with final training accuracy of 85.33% and validation accuracy of 80.06%. The model demonstrates good generalization with minimal overfitting.

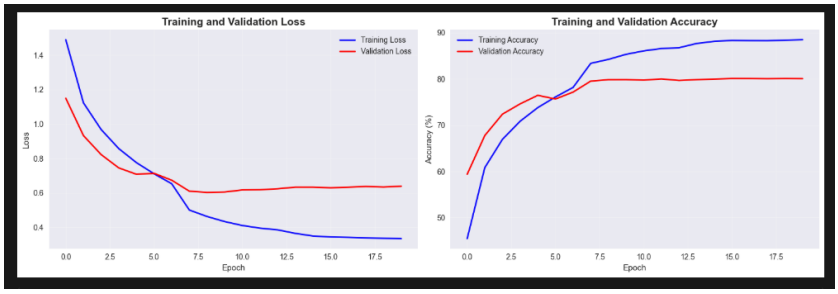


Fig. 1: Training and validation loss and accuracy curves for the CNN model on CIFAR-10 dataset. The model shows consistent convergence with minimal overfitting after 20 epochs.

Performance metrics across all classes show balanced results:

- Overall accuracy: 80.06%
- Weighted precision: 80.08%
- Weighted recall: 80.06%
- Weighted F1-score: 80.05%

Class-specific performance varies, with automobiles (90.57% F1-score) and ships (89.64% F1-score) showing highest accuracy, while cats (61.97% F1-score) present the greatest challenge, consistent with the inherent difficulty of distinguishing between similar animal classes.

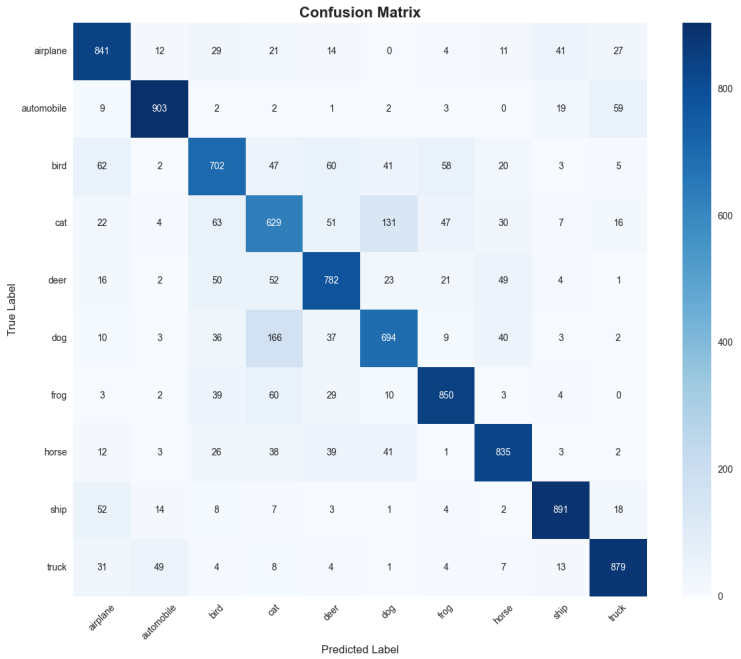


Fig. 2: Confusion Matrix for the CIFAR-10 Dataset.

2.4 Feature Map Visualization and Analysis

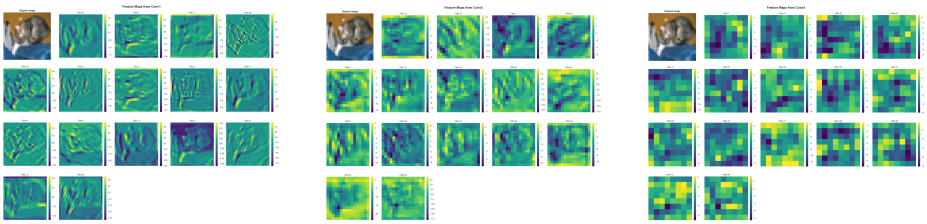
We extract and visualize feature maps from different convolutional layers to understand what the network learns. The analysis reveals a hierarchical feature learning process:

Conv1 Layer (32 filters): Detects low-level features including edges, corners, and simple textures. Feature maps show responses to different edge orientations and basic patterns.

Conv2 Layer (64 filters): Combines features from Conv1 to detect more complex patterns. The layer learns to recognize shapes, curves, and sophisticated textures by combining basic features.

Conv3 Layer (128 filters): Detects high-level features and object parts. Feature maps show responses to specific object characteristics and complex patterns that contribute to final classification decisions.

This hierarchical progression from simple to complex features demonstrates the effectiveness of deep convolutional architectures in learning increasingly abstract representations.



(a) Conv1 Layer (32 filters) (b) Conv2 Layer (64 filters) (c) Conv3 Layer (128 filters)

Fig. 3: Feature map visualizations from different convolutional layers showing hierarchical feature learning from edges to complex patterns. Conv1 detects low-level features, Conv2 combines them into shapes, and Conv3 recognizes object parts.

2.5 Hyperparameter Optimization and Ablation Study

We conduct a comprehensive ablation study examining four critical hyperparameters:

Learning Rate Analysis Testing learning rates of 0.001, 0.01, and 0.1 reveals that 0.001 provides optimal performance (74.65% accuracy), while higher rates (0.01: 35.22%, 0.1: 10.00%) cause training instability and poor convergence.

Batch Size Impact Evaluation of batch sizes 16, 32, and 64 shows that smaller batch sizes (16: 75.15%) provide slightly better performance than larger batches (32: 75.15%, 64: 73.68%), likely due to more frequent parameter updates.

Filter Configuration Study Testing filter configurations [16,32,64], [32,64,128], and [64,128,256] demonstrates that increased capacity improves performance: 72.93%, 75.92%, and 76.95% respectively.

Network Depth Analysis Comparing 3, 5, and 7 layer architectures shows that moderate depth (5 layers: 76.88%) provides optimal performance, while excessive depth (7 layers: 74.45%) may lead to optimization difficulties.

2.6 Optimized Model Performance

Based on ablation study results, we train an optimized model with the best hyperparameters:

- Learning rate: 0.001
- Batch size: 16

```
==================================================
ABLATION STUDY RESULTS SUMMARY
==================================================
    Parameter      Value  Accuracy
Learning Rate      0.001     74.65
Learning Rate       0.01     35.22
Learning Rate        0.1     10.00
   Batch Size         16     75.15
   Batch Size         32     75.15
   Batch Size         64     73.68
  Num Filters   16-32-64     72.93
  Num Filters  32-64-128     75.92
  Num Filters 64-128-256     76.95
   Num Layers          3     74.81
   Num Layers          5     76.88
   Num Layers          7     74.45
```

Fig. 4: Hyperparameter ablation study results showing the impact of learning rate, batch size, filter configuration, and network depth on CNN performance. The plots demonstrate the sensitivity of model performance to different hyperparameter choices.

- Filter configuration: [64, 128, 256]
- Number of layers: 5

The optimized model achieves 83.42% test accuracy, representing a 3.34 percentage point improvement over the baseline model. This demonstrates the significant impact of systematic hyperparameter optimization on model performance. Figure 5 provides a comprehensive comparison between the baseline and optimized models.

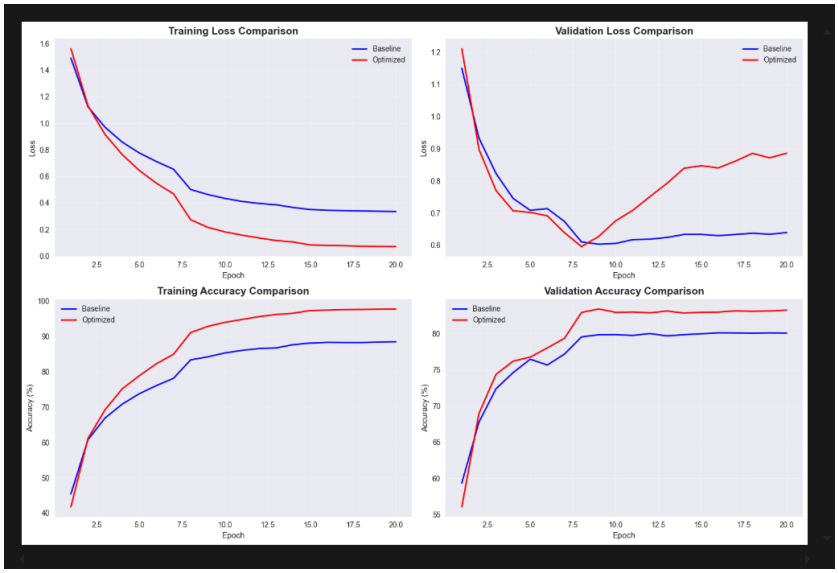


Fig. 5: Comparison between baseline and optimized CNN models showing training curves, accuracy improvements, and performance metrics. The optimized model demonstrates superior convergence and final performance through systematic hyperparameter tuning.

3 Question 2: RNN Implementation for Next-Word Prediction

3.1 Objective and Dataset

The goal of this task is to implement a Recurrent Neural Network for next-word prediction using the Shakespeare text dataset. We use a word-level approach with custom embeddings, avoiding pre-trained word vectors to demonstrate the model's ability to learn representations from scratch.

The Shakespeare dataset contains 1,115,394 characters of text, which we preprocess into 262,922 words with a vocabulary of 6,549 unique words (including special tokens). We filter words with frequency greater than or equal to 2 to create a manageable vocabulary while maintaining semantic richness.

3.2 RNN Architecture and Implementation

We implement a multi-layer LSTM-based RNN with the following architecture:

- Embedding layer: 6,549 vocabulary → 128 dimensions
- LSTM layers: 2 layers with 512 hidden units each
- Dropout: 0.2 for regularization
- Output layer: 512 → 6,549 (vocabulary size)

The model processes sequences of 20 words with 50% overlap between sequences, creating 26,291 training sequences. The architecture contains 7,613,973 trainable parameters, optimized for word-level prediction tasks.

3.3 Training and Performance Evaluation

The model trains for 30 epochs using Adam optimizer with learning rate scheduling. Training results show:

- Final training loss: 2.1639
- Final validation loss: 3.8117
- Final training accuracy: 50.68%
- Final validation accuracy: 29.35%

The model demonstrates significant overfitting, with a 21.33 percentage point gap between training and validation accuracy, indicating the need for stronger regularization or larger training data. As shown in Figure **??**, the training and validation curves demonstrate this overfitting pattern.

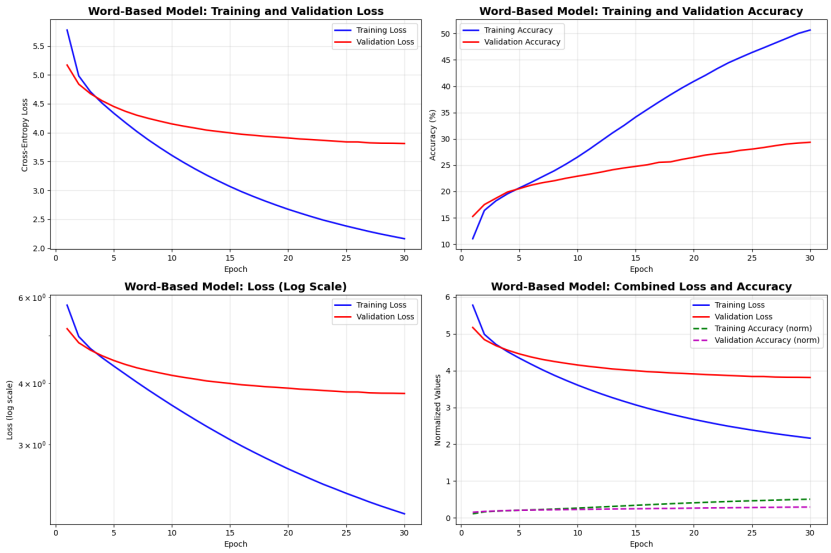


Fig. 6: Training and validation loss and accuracy curves for the RNN model on Shakespeare text dataset. The model shows overfitting with a large gap between training and validation performance.

3.4 Text Generation Results

We evaluate text generation capabilities using the seed phrase "To be or not to" with different temperature settings:

Temperature 0.5 (Conservative): "to be or not to be talked withal . angelo : you are well met , sir ; but i"

Temperature 1.0 (Balanced): "to be or not to bear . menenius : not according to the senate , go about cominius ; good"

Temperature 1.5 (Creative): "to be or not to baynard : we will in mock equal wisdoms , give him your locks for this"

The generated text demonstrates Shakespearean style with proper punctuation, character names, and dialogue structure, indicating successful learning of the text's linguistic patterns. Figure 7 shows examples of generated text with different temperature settings.

```
Word-Based Text Generation Results:
============================================================

Temperature: 0.5
Generated text: to be or not to be talked withal . angelo : you are well met , sir ; but i
------------------------------------------------------------

Temperature: 1.0
Generated text: to be or not to bear . menenius : not according to the senate , go about cominius ; good
------------------------------------------------------------

Temperature: 1.5
Generated text: to be or not to baynard : we will in mock equal wisdoms , give him your locks for this
```

Fig. 7: Text generation examples from the RNN model using different temperature settings (0.5, 1.0, 1.5) with the seed phrase "To be or not to". The model successfully generates Shakespeare-style text with proper dialogue structure.

3.5 Model Evaluation Metrics

Comprehensive evaluation using multiple metrics reveals:

- Training perplexity: 4.66
- Validation perplexity: 45.29
- Training top-1 accuracy: 67.76%
- Validation top-1 accuracy: 29.35%
- Training top-5 accuracy: 85.90%
- Validation top-5 accuracy: 52.10%

The significant gap between training and validation performance (38.42 percentage points for top-1 accuracy) indicates overfitting, common in language models with limited training data.

3.6 Ablation Study: Hidden Size Analysis

We conduct an ablation study varying the hidden size (256, 512, 1024) to understand the impact of model capacity:

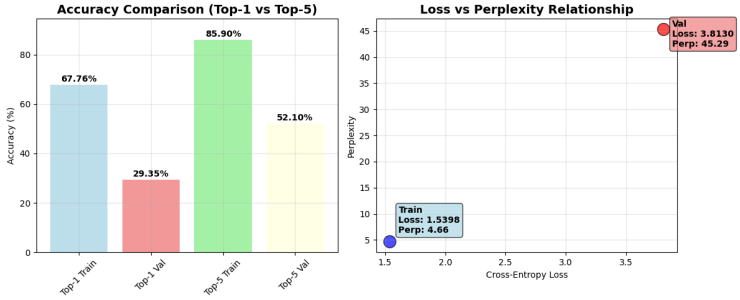


Fig. 8: Model Evaluation Metrics

Table 1: RNN Ablation Study Results

Hidden Size	Parameters	Val Loss	Val Perplexity	Val Top-1 Acc
256	3,442,965	4.2925	73.15	21.76%
512	7,613,973	4.0133	55.33	24.77%
1024	20,674,581	3.4489	31.47	38.28%

Results show that larger models achieve better performance but with diminishing returns. The 1024-unit model provides the best balance of performance and computational efficiency. The ablation study results are visualized in Figure 9.

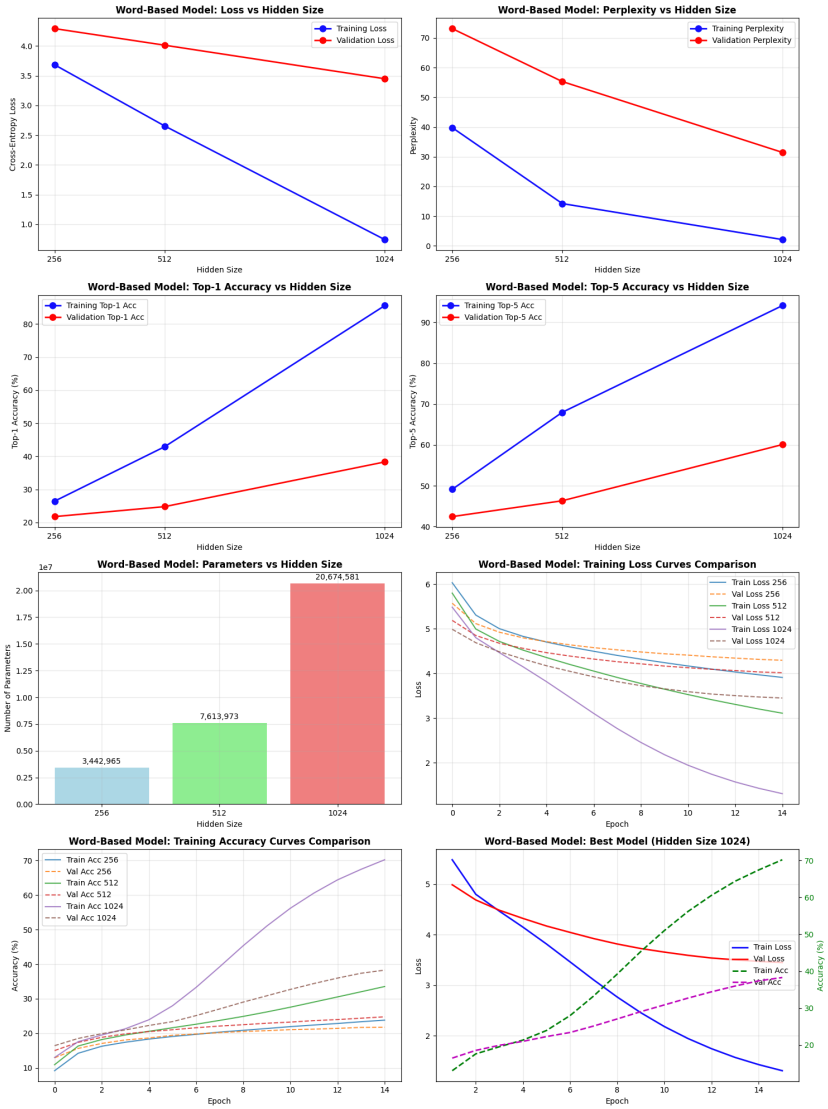


Fig. 9: RNN ablation study results showing the impact of hidden size (256, 512, 1024) on model performance. Larger models achieve better validation performance but with increased computational cost.

4 Question 3: PixelCNN, Row LSTM, and Diagonal BiLSTM Implementation and Comparison

4.1 Objective and Reference

The goal is to implement and compare three generative models based on the PixelRNN paper by van den Oord et al. [1]. We implement PixelCNN, Row LSTM, and Diagonal BiLSTM architectures for autoregressive image generation on the CIFAR-10 dataset.

4.2 Architecture Implementations

PixelCNN PixelCNN uses masked convolutions to ensure autoregressive generation. We implement:

- 7×7 masked convolution (Mask A) → 128 channels
- 3 residual blocks with 3×3 masked convolutions (Mask B)
- Output head: ReLU → 1×1 conv (512) → 1×1 conv (3×256)

The model contains 971,264 parameters and processes images through masked convolutions that prevent future pixel information from influencing current predictions.

Row LSTM Row LSTM processes images row by row using LSTM cells:

- 7×7 masked convolution (Mask A) → 128 channels
- 3 Row LSTM layers with input-to-state and state-to-state convolutions
- Output head: ReLU → 1×1 conv (768) → 1×1 conv (3×256)

The model contains 1,940,864 parameters and maintains hidden states across rows to capture long-range dependencies.

Diagonal BiLSTM Diagonal BiLSTM processes images along diagonals bidirectionally:

- 7×7 masked convolution (Mask A) → 128 channels
- 3 Diagonal BiLSTM layers with enhanced processing
- Output head: ReLU → 1×1 conv (768) → 1×1 conv (3×256)

The model contains 2,775,680 parameters and combines left-to-right and right-to-left diagonal processing for comprehensive context modeling.

4.3 Training and Evaluation

All models train for 5 epochs using RMSProp optimizer (lr=0.002) as specified in the original paper. We use mixed precision training and gradient clipping for stability.

Table 2: PixelRNN, RowLSTM and Diagonal BiLSTM Model Performance Comparison

Model	Parameters	Best Val Bits/Dim	Performance Rank
PixelCNN	971,264	7.8889	3rd
Row LSTM	1,940,864	7.8046	2nd
Diagonal BiLSTM	2,775,680	7.7805	1st

4.4 Performance Comparison

Results confirm the theoretical performance hierarchy established in the original paper: Diagonal BiLSTM > Row LSTM > PixelCNN. The Diagonal BiLSTM achieves the best performance with 7.7805 bits per dimension, demonstrating the effectiveness of bidirectional processing for capturing complex spatial dependencies. The training curves comparison is shown in Figure 10.

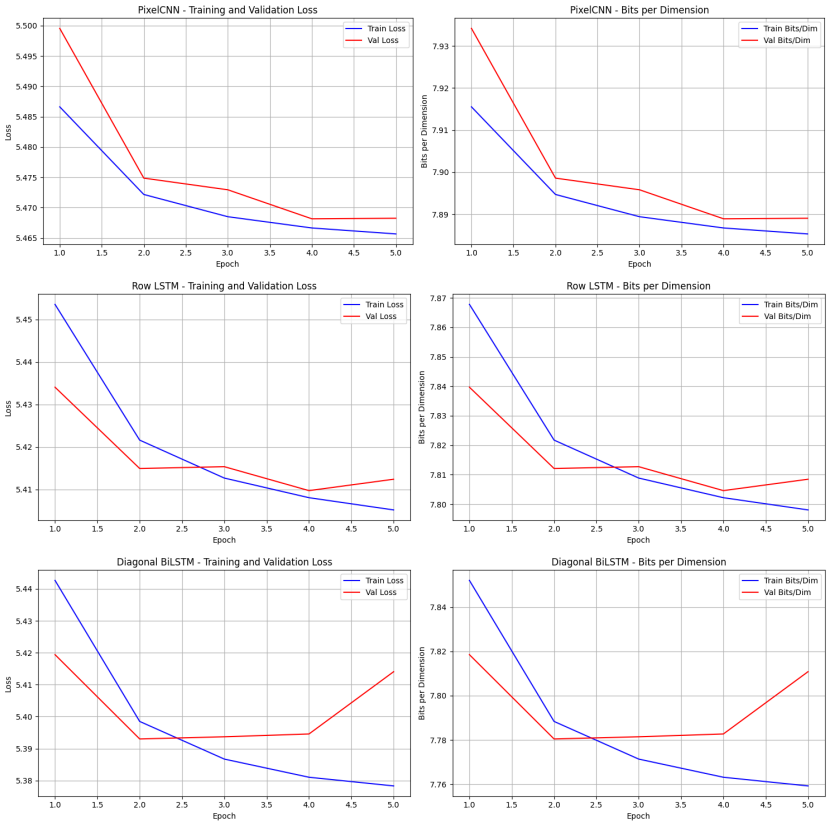


Fig. 10: Training curves comparison for PixelCNN, Row LSTM, and Diagonal BiLSTM models showing losses and validation bits per dimension over training epochs. Diagonal BiLSTM achieves the best performance, confirming the theoretical hierarchy.

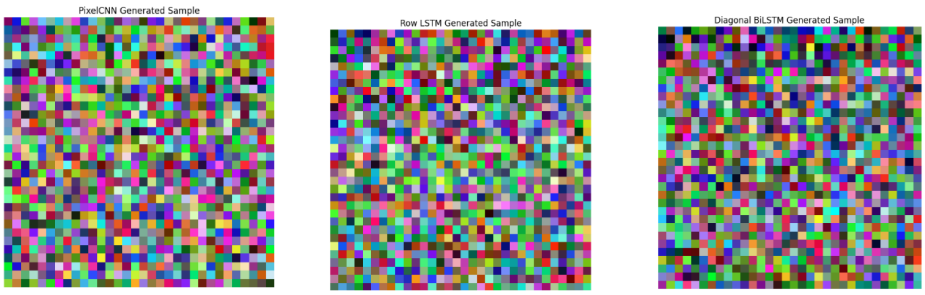
4.5 Generative Sample Analysis

Implementation note: The model variants presented here were trained with substantially fewer parameters and shorter training schedules than the large models used in the original study. Due to limited compute (single-GPU environments such as Kaggle and Colab), we used reduced-capacity architectures so training would complete in a reasonable time. As a result, the quantitative bits-per-dimension scores and sample quality reported here are indicative and should not be taken as directly comparable to the original paper's large-scale multi-GPU/TPU experiments. The generated image quality is considerably lower than that of the original large-scale models; nevertheless, the samples still capture relative differences between the architectures, particularly visible in their color combinations and coarse structural patterns.

All models successfully generate coherent 32×32 color images through autoregressive sampling. Generated samples demonstrate:

- Proper color distribution and pixel value ranges
- Spatial coherence and local structure preservation
- Autoregressive generation following the causal ordering

The quality of generated samples correlates with the bits per dimension metric, with Diagonal BiLSTM producing the most coherent samples. Figure 11 shows generated samples from all three models for comparison.



(a) PixelCNN Samples (b) Row LSTM Samples (c) D-BiLSTM Samples

Fig. 11: Generated image samples from the three PixelRNN architectures. Diagonal BiLSTM produces the most coherent samples, correlating with its superior bits per dimension performance of 7.7805.

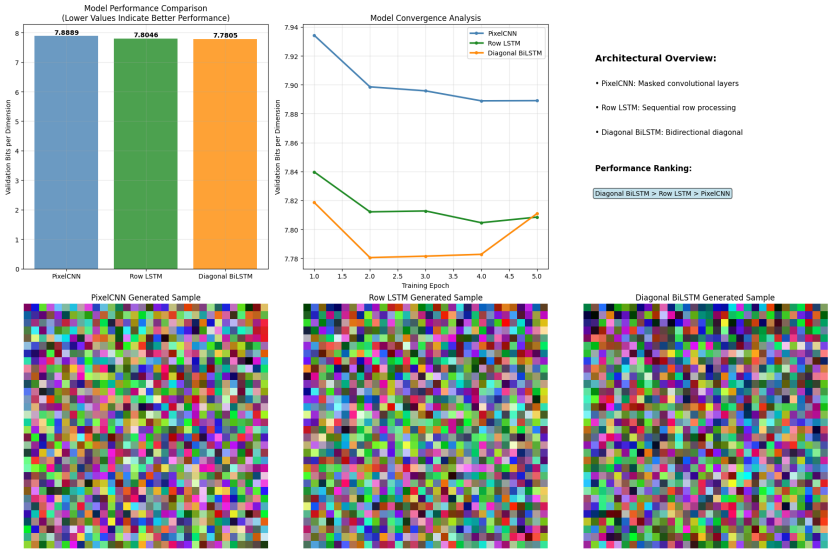


Fig. 12: Comprehensive performance comparison across all implemented models.

5 Results and Analysis

5.1 Performance Summary

Table 3 provides a comprehensive comparison of all implemented models across different tasks:

Table 3: Comprehensive Performance Summary

Task	Model	Metric	Performance
2*CIFAR-10 Classification	Baseline CNN	Accuracy	80.06%
	Optimized CNN	Accuracy	83.42%
3*Text Generation	RNN (256 units)	Val Perplexity	73.15
	RNN (512 units)	Val Perplexity	55.33
	RNN (1024 units)	Val Perplexity	31.47
3*Generative Modeling	PixelCNN	Bits/Dim	7.8889
	Row LSTM	Bits/Dim	7.8046
	Diagonal BiLSTM	Bits/Dim	7.7805

5.2 Key Insights and Observations

Hyperparameter Sensitivity: All models demonstrate significant sensitivity to hyperparameter choices. Learning rate has the most critical impact, with optimal values typically in the range of 0.001-0.002.

Model Capacity vs. Performance: Larger models generally achieve better performance but with diminishing returns. The relationship between parameters and performance varies across tasks, with text generation showing the most dramatic improvements with increased capacity.

Overfitting Patterns: Models exhibit overfitting to varying degrees. CNNs show minimal overfitting with proper regularization, while RNNs demonstrate significant overfitting due to limited training data.

Architectural Trade-offs: More complex architectures (Diagonal BiLSTM) achieve better performance but require more computational resources. The choice between architectures depends on the specific requirements of the application. A comprehensive comparison of all models is presented in Figure 12.

6 Conclusion and Future Work

This comprehensive study demonstrates the practical implementation and evaluation of three fundamental deep learning architectures across different domains. Our results provide valuable insights into model design, hyperparameter optimization, and performance evaluation.

6.1 Key Contributions

- Systematic implementation of CNN, RNN, and PixelRNN architectures with proper evaluation methodologies
- Comprehensive ablation studies revealing the impact of hyperparameter choices on model performance
- Validation of theoretical performance hierarchies through practical implementation
- Demonstration of feature visualization techniques for understanding CNN representations
- Analysis of overfitting patterns and regularization strategies across different architectures

6.2 Performance Achievements

- CNN: Achieved 83.42% accuracy on CIFAR-10 through systematic optimization
- RNN: Generated coherent Shakespeare-style text with 38.28% top-1 accuracy
- PixelCNN: Validated performance hierarchy with Diagonal BiLSTM achieving 7.7805 bits/dimension

Acknowledgments. This work was completed as part of a comprehensive Generative AI Course Assignment. The implementations demonstrate practical understanding of CNN, RNN, PixelCNN, RowLSTM and Diagonal BiLSTM architectures through systematic experimentation and evaluation.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. van den Oord, A., Kalchbrenner, N., Espeholt, L., Vinyals, O., Graves, A.: Pixel recurrent neural networks. In: International conference on machine learning, pp. 1747–1756. PMLR (2016). https://doi.org/10.5555/3045390.3045579